Clearly, we'll get a fatal error if we try to run this. There is no function called `wrongFunc()` declared. However, there is nothing wrong with the syntax of this code. The error only emerges when it is executed. If we were to attempt to call the non-existent `wrongFunc()` by name it would be flagged by your IDE as you wrote it, and a static analysis tool like `phpmd` (PHP Mess Detector, a very useful tool for monitoring code quality available here: https://phpmd.org/) would also generate a warning. The variable call is far more likely to slip through whatever layers of analysis are in place to protect your codebase (in fact only a unit test would expose the issue). Use variable functions sparingly, they are best limited to call-backs handling where type declarations can enforce some checks and balances.

Anonymous Functions

There are occasions when functions don't need to be named, because of the way that they are called. Sometimes other functions require that they're given something callable as an argument, typically to be executed when the function is complete or needs to raise an error. These passed-in functions are known as *callbacks*. If you never need to call a function by its identifier (because another function will do the call for you), why bother naming it all?

When a function isn't given a name, it is an *anonymous function*. Another name for anonymous functions is *closure*, these are very commonly encountered in other languages (JavaScript, for example) and were added to PHP version 5.3. The name *Closure* is important in PHP because anonymous functions are implemented internally as objects of the built-in *Closure* class.

Here is how to declare an anonymous function:

```
function (string $txt): void {
    print $txt . PHP_EOL;
};
```

The above code is legal PHP syntax, but not particularly useful on its own. Most IDEs will likely highlight this code in some way because it is useless, there's no way to call this code.

Up to now we have been using function names explicitly to invoke them, but any expression that evaluates to a function or function name can also be used. The above code evaluates to a function, and we can call it using the standard calling mechanism